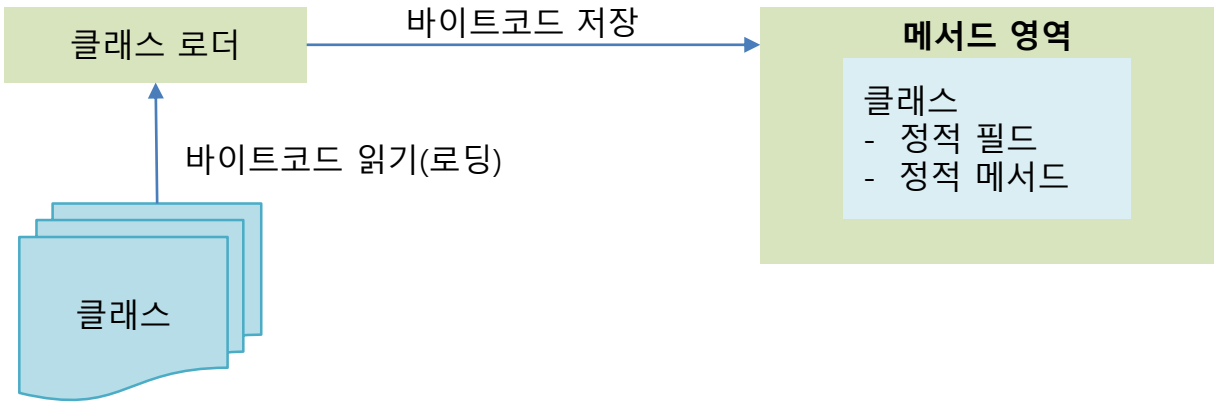
11강. static

- static의 이해
- static 멤버 선언 및 사용
- static 블록
- static 멤버와 인스턴스 멤버 관계

11-1. static의 이해

static(정적) 키워드의 이해

➢ 자바는 클래스 로더(loader)를 이용해서 클래스를 메서드 영역에 저장하고 사용한다. 정적(static) 멤버란 메서드 영역의 클래스에 고정적으로 위치하는 멤버를 말한다. 정적(static) 멤버는 해당 클래스의 모든 인스턴스에서 공유되며, 클래스가 로드될 때 한번 초기화되고 프로그램이 종료 될 때가지 유지됩니다. 정적(static) 멤버는 객체를 생성할 필요없이 클래스를 통해 바로 사용 가능하다.



- 정적(static) 키워드의 장점은 메모리 효율성과 코드의 간결성을 높일 수 있다
- 단점은 과용할 경우 결합도가 높아질 수 있음(적절한 사용 필요)

11-1. static 멤버 선언 및 사용

static(정적) 멤버 선언

- 필드와 메서드는 모두 정적(static)멤버가 될 수 있다.
- 정적 필드와 정적 메서드로 선언하려면 static 키워드를 추가하면 된다.

```
public class 클래스 {
  // 정적 필드 선언
  static 타입 필드 [= 초기값];

  // 정적 메서드
  static 리턴타입 메서드( 매개변수, ... ) { ... }
}
```

- 객체마다 가지고 있을 필요성이 없는 공용(공통) 필드는 정적(static) 필드로 선언하는 것이 좋다.
- Calculator 클래스에서 원의 넓이나 둘레를 구할 때 필요한 파이는 Calculator 객체마다 가지고 있을 필요가 없기 때문에 정적(static) 필드로 선언하는 것이 좋다.

```
public class Calculator {
    String color;                  // 계산기별로 색상이 다를 수 있다.
    static double pi = 3.14159; // 계산기에서 사용하는 PI 값은 동일
}
```

11-1. static 멤버 선언 및 사용

static(정적) 멤버 선언

- 인스턴스 필드를 이용하지 않는 메서드는 정적 메서드로 선언하는 것이 좋다.
- 아래 예에서 Calculator의 plus() 메서드는 외부에서 주어진 매개 값들을 가지고 처리하므로 정적 메서드로 선언하는 것이 좋다. 그러나 인스턴스 필드인 color를 변경하는 setColor() 메서드는 인스턴스 메서드로 선언해야 한다.

```
public class Calculator {
    String color;          // 인스턴스 필드

    // 인스턴스 메서드
    void setColor(String color) {
        this.color = color;
    }

    // 정적 메서드
    static int plus( int x, int y ){
        return x + y;
    }

    // 정적 메서드
    static int minus( int x, int y ) {
        return x - y;
    }
}
```

11-1. static 멤버 선언 및 사용

static(정적) 멤버 사용 (java_11_1_ex1_static)

- 클래스가 메모리에 로딩되면 정적 멤버를 바로 사용할 수 있는데, 클래스 이름과 함께 도트(.) 연산자로 접근하면 된다.

(작성 예)

```
public class Calculator {
  static double pi = 3.14159;
  static int plus( int x, int y ) { return x + y; }
  static int minus( int x, int y ) { return x – y; }
}
```

(사용 예)

```
double result1 = 10 * 10 * Calculator.pi;
Int result2 = Calculator.plus(10, 5);
Int result3 = Calculator.minus(10, 5);
```

- 정적 필드와 정적 메서드는 아래처럼 객체 참조 변수로도 접근이 가능하나 정적 요소는 클래스 이름으로 접근하는 것이 정석이다.

```
Calculator calcu = new Calculator();
double result1 = 10 * 10 * calcu.pi;
int result2 = calcu.plus(10, 5);
int result3 = calcu.minus(10, 5);
```

11-1. static 블록

static(정적) 블록

- 정적 필드는 아래와 같이 선언과 동시에 초기 값을 주는 것이 일반적이다.

```
static String company = "Samsung";
static String model = "NoteBook";
static String info;
```

- 복잡한 초기화 작업 필요하면 정적 블록(static block)을 사용한다.
- 정적 블록은 클래스가 메모리로 로딩될 때 자동 실행되며, 생성자에서 초기화를 하지 않습니다.
- 정적 블록이 여러 개가 선언된 경우 선언된 순서대로 실행된다.

```
static String company = "Samsung";
static String model = "NoteBook";
static String info;

// 정적 블록
static {
    info = company + "-" + model;
}
```

11-1. static 멤버와 인스턴스 멤버 관계

static(정적) 인스턴스 멤버 사용 불가

- 정적 메서드와 정적 블록은 객체가 없어도 실행된다는 특징때문에 내부에 인스턴스 필드나 인스턴스 메서드를 사용할 수 없다.
- 객체 자신의 참조인 this도 사용할 수 없다.

```
String color;                          // 인스턴스 필드
static String company = "Samsung";     // 정적 필드
static String model = "NoteBook";      // 정적 필드
static String info;                    // 정적 필드

void powerOn() {
    System.out.println("전원을 켭니다.");
}

static int plus(int x, int y) {
    return x + y;
}
// 정적 블록
static {
    this.color = "검정";               // X: this 사용불가
    powerOn();                         // X: 인스턴스 메서드
    info = company + "-" + model;      // O: 정적 필드 가능
    plus(10, 20);                      // O: 정적 메서드 가능
}
```

11-1. static 멤버와 인스턴스 멤버 관계

static(정적) 인스턴스 멤버 사용 불가

- 정적 메서드와 정적 블록에서 인스턴스 멤버를 사용하고 싶다면 객체를 생성하고 참조 변수로 접근을 해야 한다.

```
int speed;                          // 인스턴스 필드 선언

void run() {                        // 인스턴스 메서드 선언
    System.out.println(speed + "로 달립니다.");
}

static void simulate() {            // 정적 메서드 선언
    Car myCar = new Car();          // 객체 생성

    myCar.speed = 60;
    myCar.run();
}

public static void main(String[] args) {
    // 정적 메서드 호출
    simulate();

    Car myCar = new Car();

    // 인스턴스 멤버 사용
    myCar.speed = 100;
    myCar.run();
}
```

11-1. static 멤버와 인스턴스 멤버 관계

static(정적) 인스턴스 멤버 사용 불가

- main() 메서드도 정적 메서드이므로 객체 생성없이 인스턴스 필드와 인스턴스 메서드를 사용할 수 없다.

```
public class Car {
  int speed;                                  // 인스턴스 필드 선언
  int void run() { ... };                     // 인스턴스 메서드 선언

  public static void main(String[] args) {
    speed = 60;                     (X) > 컴파일 오류
    run();                          (X) > 컴파일 오류
}
```

➢ 예제 1

println() 메서드는 매개값을 콘솔에 출력합니다. println() 메서드의 매개값으로 는 int, Boolean, double, String 타입의 값을 줄 수 있습니다. 객체 생성없이 호출할 수 있도록 다음 조건과 예제 코드를 보고 Printer 클래스에서 println() 메서드를 오버로딩하여 선언해보세요.

리턴 타입	메서드 이름	매개변수(타입)
void	println	value

```
Printer.println(10);
Printer.println(true);
Printer.println(5.7);
Printer.println("홍길동");
```